Domain: Multi-Core / Many-Core Architectures

Distributing Uneven Workloads Across Unequal Multi-Core Machines: Introduction and Research Methodology

Ahsan Baig — 23L-0515

Sundas Habib — 23L-2580

Department of Computer Science, FAST NUCES, Lahore

I. Introduction

The speed of a single processor core has grown only slowly for many years. Most of the extra performance in new computers now comes from having more cores, and recent chips go a step further by mixing cores of different kinds. Intel's 12th-generation processors, for example, combine fast "performance" cores with smaller "efficient" cores on one chip. Programs therefore have to divide their work between cores that are not equal, and the old assumption that every core runs at the same speed no longer holds.

The same change is happening beyond a single chip. A few ordinary laptops can now be linked into a small cluster at almost no cost, which is attractive for students, small research groups and home users. Such a cluster is almost never uniform. Laptops bought a few years apart differ in core count, core design, memory and cooling. Even one laptop does not keep a steady speed: it runs faster for a short time when it is cool, then slows down once it heats up or reaches its power limit, and the operating system moves work between its fast and slow cores while a program runs.

Some kinds of work make this problem much harder. In game solving, the aim is to find the result of a game when both players play perfectly. The search behind this is very uneven: one move may be settled almost at once while a neighbouring move needs hours of searching, and there is usually no way to tell which is which before the search is done. Sharing such work fairly is difficult even on identical machines. On unequal machines whose speed changes over time, new problems appear. A slow machine may be left holding one very large task while a faster machine sits idle. A plan based on each machine's speed at the start becomes wrong as the machines warm up. And the more finely the work is cut to keep machines busy, the more likely it is that the same positions are searched twice.

In our literature review, we examined four recent studies that touch on this problem from different sides [1]–[4]. They fall into two groups. The game-solving systems [1], [2] deal with very uneven work, but run on identical, powerful servers. The load-balancing studies [3], [4] deal with machines or cores of different speeds, but only with even, predictable work. From this comparison we identified five research gaps:

1. Real differences between machines are avoided or simulated.
2. Machine speed is treated as fixed during a run.
3. Uneven work and unequal machines have not been studied together.
4. Everyday hardware and networks are left out.
5. Repeated work in small, unequal setups is not measured.

This paper addresses these gaps together. Our research problem is: how can unpredictable, uneven work be shared efficiently between a small number of unequal consumer machines whose speed changes while they work, and how much work is repeated when it is shared?

To study this problem, we build a distributed Connect Four solver and run it on two Windows laptops of different processor generations: one with a 12th-generation hybrid Intel Core i7, and one with an 8th-generation Intel Core i5. The laptops are connected either through an ordinary Wi-Fi network or through a direct Ethernet cable. We compare four ways of sharing the work, starting from a fixed split decided before the run and ending with dynamic sharing that also splits long tasks, cancels tasks that are no longer needed, and shares results that have already been found. While the solver runs, we record how long the work takes, how long each laptop waits, how much work is repeated, and how each laptop's clock speed, temperature and power change over time.

The planned contributions of this work are:

- An experimental study of uneven work and unequal machines together, on real consumer laptops, with no simulated differences between them.
- A step-by-step comparison of four work-sharing strategies, in which each strategy adds one group of ideas to the one before, so that the value of each group can be seen.
- Measurements that link the changing speed of each laptop during a run to how well the work stays balanced.
- A measure of how much work is repeated when one search is split between two unequal machines, and of how the size of the split and the sharing of results change it.

Every result is checked against published Connect Four results [5], [6], so that faster runs never come at the cost of wrong answers. The rest of this document is organised as follows. Section II sets out our research methodology: the research questions, the experimental platform, the workload, the system design, the work-sharing strategies, the experiments, the metrics, the procedure, the correctness checks, the analysis and the threats to validity. Implementation details and results will follow in the complete paper.

II. Research Methodology

This section explains how we will answer the research problem stated in Section I. Our study is built on measurement. We build one distributed game solver, run it on two real laptops of different processor generations, and compare four ways of sharing the work between them under conditions we keep as steady as we can. Each research gap becomes a research question, and each question is answered by a specific experiment and a specific set of measurements.

A. Research Questions

Table I links each research gap to the question we ask about it and to the way we answer it.

TABLE I: Mapping of Research Gaps to Research Questions

Gap	Research question	How we answer it
Gap 1: Real differences between machines are avoided or faked	RQ1: How well can uneven work be shared between two machines that really differ in processor generation, core design, memory and cooling, and how much do the fast and slow cores inside Laptop 1 each contribute?	Two real laptops with no simulated slowdown; each laptop also solves the problem alone; the work done by Laptop 1's two core types is logged separately
Gap 2: Machine speed is treated as fixed	RQ2: How much does each laptop's speed change during a run, and how does that change affect a split decided at the start compared with sharing work during the run?	Sensor logs every second; a speed-based fixed split is compared with dynamic sharing; the speed measured at the start is compared with the speed actually reached
Gap 3: Uneven work and unequal machines have not been studied together	RQ3: When task sizes are unknown and machines are unequal at the same time, which sharing strategy finishes soonest and leaves the least time unused?	A game-solving workload with very uneven tasks, run with four strategies of increasing ability (V0 to V3)
Gap 4: Everyday hardware and networks are left out	RQ4: Does an ordinary shared Wi-Fi network, compared with a direct cable, change the result?	Every strategy is run over Wi-Fi and over a direct Ethernet cable; network delay, speed and waiting time are measured on both
Gap 5: Repeated work in small, unequal setups is not measured	RQ5: How much extra work is caused by splitting the problem between two unequal machines, and how do the size of the split and the sharing of solved results change it?	All positions searched by both laptops are counted and compared with single-laptop and unsplit references; the split depth is varied; strategies with and without shared results are compared

B. Experimental Platform

Machines. We use two laptops that differ in almost every way that matters for Gap 1 (Table II). Laptop 1 has a hybrid processor that mixes fast "performance" cores and slower "efficient" cores on the same chip, so it also differs inside itself. Laptop 2 has a processor four generations older, with identical cores. It is also a low-power laptop chip, and is allowed far less processor power than Laptop 1. Neither machine is slowed down on purpose: every difference comes from the hardware itself.

TABLE II: Experimental Platform

	Laptop 1	**Laptop 2**
Model	Dell Precision 5570	HP EliteBook 840 G5
Processor	Intel Core i7-12800H (12th gen): 6 performance + 8 efficient cores, 20 threads	Intel Core i5-8350U (8th gen): 4 cores, 8 threads
Power limits (sustained / boost)	40 W / 115 W	18 W / 44 W
Memory	32 GB DDR5-4800	16 GB DDR4-2400
Operating system	Windows 11 Pro 26H2	Windows 11 Pro 25H2
Network links	Intel Wi-Fi 6E AX211; USB-C gigabit Ethernet adapter	Intel Dual Band Wireless-AC 8265; built-in Intel I219-LM gigabit Ethernet
Power settings	Plugged in; best performance mode; Dell "Ultra Performance" profile	Plugged in; best performance mode

Software. Both laptops run the same programs, built from the same source code with the same compiler (GCC 16.1.0) and the same settings. The build uses an instruction set that both processors support (the AVX2 level), so neither laptop gets code tuned for its own chip. Every program writes the exact source version, compiler and build settings at the top of its log, so each result can be traced to the code that produced it. Tool versions and system settings of both laptops are recorded in a shared environment file and are not changed while experiments are running.

Network. The laptops are linked in two ways for Gap 4: through an ordinary shared Wi-Fi network (a home router or a phone hotspot), and through a direct gigabit Ethernet cable with fixed addresses and no router in between. The same Wi-Fi network is used for every Wi-Fi run and is described with the results. In cable runs, the system connects only through the cable addresses; Wi-Fi stays connected, so that the background state of both laptops is the same in both kinds of run.

C. Workload: Solving Connect Four

Why Connect Four. We need a workload whose task sizes are very uneven and cannot be known in advance (Gap 3), and whose correct answer is known, so that every run can be checked. Connect Four meets both needs. The standard 7×6 game was first solved in 1988 and is a win for the first player, and results for many other board sizes have been published [6]. Pons has published a detailed guide to building a solver, together with six test sets of 1,000 positions each and the correct score of every position [5]. The game tree is also very lopsided: some positions are decided after searching only a handful of later positions, while others need a very large search. The spread of task sizes that we measure will be reported with the results.

Solver. Each laptop runs a solver that follows the approach described by Pons [5]. It searches ahead on the assumption that both players play perfectly (negamax search with alpha-beta pruning, which skips moves that cannot change the outcome), stores the board in a compact form using 64-bit numbers, remembers positions it has already solved in a transposition table, and checks the most promising moves first. The solver only decides whether the player to move wins, draws or loses, without working out how quickly. Pons calls this a weak solver [5]. It is enough to find the game's result, it needs only a very narrow score range in the search, and it allows the early stopping described in Section II-E.

Inside each laptop, the solver runs one thread per logical processor (20 on Laptop 1 and 8 on Laptop 2). The threads take tasks from one shared pile and share one transposition table, which is kept for the whole run. All of this is the same in every strategy.

Turning the game into tasks. The coordinator expands the game tree from a chosen start position down to a fixed depth d, the split depth. Every position at depth d becomes one task, written as the sequence of columns played to reach it (for example "4453"). The solver returns win, draw or loss for the task, and the coordinator combines these answers back up the tree to obtain the result of the start position. Some tasks finish in milliseconds and others take minutes, and nobody knows in advance which are which.

Different move orders can lead to the same position, so some tasks are copies of each other. From the empty board, for example, the 343 move sequences of length three lead to only 238 different positions, and the 2,401 sequences of length four lead to only 1,120. At depth two there are no copies. For this reason we use split depths of three and four. Positions that are mirror images of each other are treated as different tasks.

Problem size. The start position is chosen in the pilot study (Section II-F) so that Laptop 1 alone needs a few minutes to solve it, several times longer than the time its clock speed takes to settle after the first boost. This keeps the slow-down we study in Gap 2 inside every run, while leaving enough time to repeat every run. The chosen position, its known result and its single-laptop solve times are reported with the results.

D. System Design

The system has three parts, shown in Fig. 1.

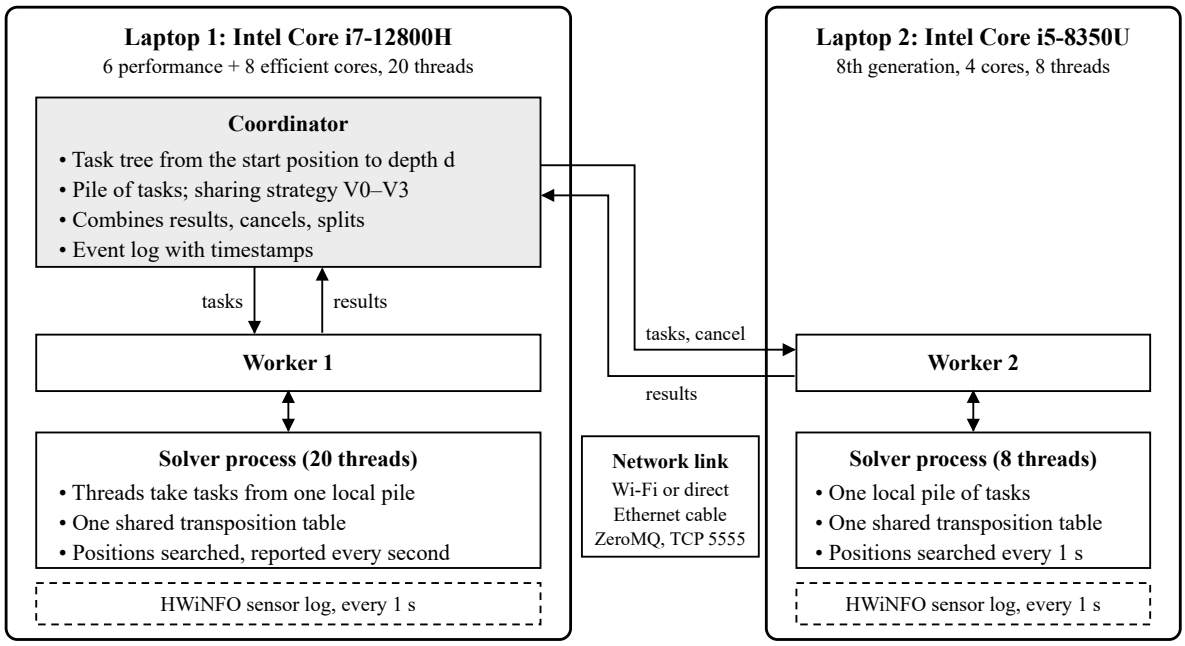


Fig. 1: System architecture. The coordinator on Laptop 1 holds the task tree and hands out tasks; a worker on each laptop drives a multi-threaded solver; the two laptops talk over Wi-Fi or a direct cable.

The **coordinator** runs on Laptop 1. It builds the task tree, decides which laptop gets which task according to the strategy under test, combines results as soon as they arrive, and records every event with a timestamp: task sent, started, finished, stopped, split or cancelled. In every strategy, a run ends as soon as the result of the start position is known.

One **worker** runs on each laptop, including Laptop 1, so both laptops are treated the same way. The worker receives tasks, passes them to its solver, and sends back each result with the number of positions searched and the time taken. Messages are a few bytes long and travel over a standard messaging library (ZeroMQ). Each worker keeps at most one waiting task per solver thread, so that a laptop cannot hold on to tasks it has not started.

Each worker starts one **solver process** at the beginning of a run and keeps it until the end, so start-up costs are paid once. While a task runs, the solver reports its running count of positions searched every second, which lets us follow each laptop's speed during long tasks.

E. Work-Sharing Strategies

We compare four strategies. Each adds one group of related ideas to the one before it (Table III). Where a group holds more than one idea, we separate their effects using counters recorded in every run: work lost in stopped tasks, work skipped through cancelling, and tasks answered from stored results.

V0: Fixed split based on measured speed. Just before each V0 run, both laptops solve the same short calibration set of positions and their speeds are measured. The coordinator then deals the tasks out in a fixed order, in turn, in proportion to the two speeds, once, before the solve begins. No work moves between the laptops after that. This is a careful but fixed plan, similar to the speed-measured split in [4]. Because task sizes are unknown, it can only balance the number of tasks, not their true cost, and the speed it relies on is measured while the laptops are still cool (Gaps 2 and 3). Calibration time is reported separately and is not part of the solve time.

V1: Ask for more work. Nothing is divided in advance. The coordinator keeps every task in one pile, and a laptop asks for more whenever it runs low. The faster laptop therefore takes more tasks without its speed ever being measured, and a laptop that slows down simply asks less often.

V2: V1 plus splitting long tasks and cancelling unneeded ones. If a task runs longer than a time limit chosen in the pilot study, it is stopped and replaced by its child positions one move deeper, which go back to the pile. A large task held by one laptop can then be shared by both. In addition, as soon as one move from a position is found to win for the player to move, that position is decided. The coordinator then cancels all tasks below it, whether waiting or running, and the decision may pass further up the tree and settle larger parts of it.

V3: V2 plus sharing solved results. The coordinator remembers every result it receives, merges tasks that are the same position into one, and answers repeated positions without sending them out again. Where possible, it also sends tasks that share a parent position to the same laptop, so that the transposition table on that laptop, which already holds related positions, can be reused. This follows the finding in [1] that keeping and sharing results greatly reduces repeated work. If V3 cannot be completed in the time available, results will be reported for V0 to V2, and this will be stated.

TABLE III: Work-Sharing Strategies

Strategy	Split decided	Work moves between laptops	Long tasks split	Unneeded tasks cancelled	Solved results shared
V0	Before the run, by measured speed	No	No	No	No
V1	During the run, on request	Yes	No	No	No
V2	During the run, on request	Yes	Yes	Yes	No
V3	During the run, on request	Yes	Yes	Yes	Yes

F. Experiments

The experiments are grouped as follows (Table IV).

E0: Pilot study. A few trial runs fix the open settings before any measured run: the start position, the split depth used in E2, the time limit for V2 and V3, the calibration set for V0, and the cool-down rule (Section II-H). Pilot runs check the setup only and are not reported as results.

E1: References. Each laptop solves the problem alone, with one worker and every feature of V3 that also helps a single machine (cancelling, splitting long tasks and merging copies). This is the best our solver can do on one laptop, so any speedup in E2 comes from adding the second laptop and not from features a single laptop could use as well. These runs also show how each laptop's speed changes over a run when it works alone. In addition, Laptop 1 solves the start position once as a single search on one thread, without any splitting. Only the number of positions searched is used from this run, as the unsplit reference for RQ5.

E2: Main comparison. All four strategies are run on both laptops together, over Wi-Fi and over the cable, at the split depth chosen in the pilot (RQ1 to RQ4).

E3: Split depth. V1 and V3 are run again over the cable at the other split depth, to show how task size affects balance and repeated work, with and without shared results (RQ5).

Network measurements. At the start of every session, the round-trip delay and transfer speed of both links are measured with standard tools (ping and iperf3), together with the round-trip time of a short message through the messaging library the system uses (RQ4).

TABLE IV: Experiment Plan

Set	What changes	Measured runs
E0 Pilot	Settings being tuned	Not reported
E1 References	Laptop (1 or 2)	2 × 3 repeats = 6, plus 1 unsplit count
E2 Main	Strategy (V0–V3) × link (Wi-Fi, cable)	4 × 2 × 3 repeats = 24
E3 Split depth	Strategy (V1, V3) at the second depth, cable	2 × 3 repeats = 6
Network	Link (Wi-Fi, cable)	Once per session

Every measured run is repeated three times. The factors we vary are the strategy, the network link, the split depth and, in E1, the laptop. Everything else is held fixed: the start position, the program build, the number of solver threads, the transposition table size, the power settings, the starting thermal state and the background programs.

G. Metrics

Table V lists what we measure and how each value is calculated. T_1 and T_2 are the E1 solve times of Laptop 1 alone and Laptop 2 alone.

TABLE V: Metrics

Metric	Definition	RQ
Solve time (T)	Time from the first task being sent until the result of the start position is known	All
Speedup	$S_1 = T_1 / T$ and $S_2 = T_2 / T$	1, 3
Combined efficiency	$E = T_ideal / T$, where $T_ideal = (T_1 \times T_2) / (T_1 + T_2)$ is the time the two laptops would need if they shared the E1 amount of work perfectly. E = 1 means perfect sharing. E can go above 1 if a shared run happens to avoid work that the single-laptop runs did, so positions searched are always reported next to E	1, 3
Idle thread-time	For each laptop, (number of threads × T) minus the total busy time of all its solver threads; reported in total and for the period after the laptop's last task ends	3
Core-type share (Laptop 1)	Share of busy time and average clock of the performance cores and of the efficient cores, from the per-core sensor log	1
Extra work	Total positions searched by both laptops, including stopped and cancelled tasks, divided by the positions searched in the matching reference: Laptop 1 alone (E1), to show the cost of sharing, and the unsplit search (E1), to show the cost of splitting. A value of 1.3 means 30% more work than the reference	5
Copied tasks	Number of tasks that were the same position as another task, and the positions searched in them	5
Lost and saved work	Positions searched in tasks that were later stopped or cancelled, and the number of tasks skipped through cancelling or answered from stored results	5
Speed over time	Positions searched per second by each laptop, in 10-second windows, read together with its clock and power	2
Clock, temperature and power	For each laptop, every second: average effective core clock (the speed the cores actually ran at), package temperature and power (for the whole processor), and the sensor flags that show when the processor slowed itself down because of heat or power limits	2
Calibration error (V0)	The ratio of Laptop 1's speed to Laptop 2's speed measured before the run, compared with the same ratio reached over the run	2
Network cost	Round-trip delay and transfer speed of each link; per run, the number of messages and bytes sent, and the time workers spent waiting for replies as a share of T	4

The number of positions searched does not depend on clock speed, but it does change slightly with the number of threads and with what the shared table already holds. We therefore compare these counts only between runs with the same setup, and we report their spread across repeats.

H. Data Collection and Procedure

Logging. Each run produces four kinds of raw records: the coordinator's event log, each worker's task log, each solver's per-second position counts, and a sensor log from HWiNFO on each laptop, written every second. Durations are measured with a clock that never jumps, on each machine separately. To line up the two laptops' sensor logs with the event log, both clocks are synchronised with an internet time server at the start of each session, and the coordinator also measures the clock difference to Laptop 2 at the start and end of every run using timestamped messages. Raw logs are never edited; every table and chart is produced from them by scripts.

Before each session, on both laptops: the charger is plugged in, Windows Update is paused, the clocks are synchronised, all other programs are closed, the power settings are checked, and the room temperature is noted.

Before each run, both laptops rest idle for a minimum time and until each processor's temperature falls below a level chosen for that laptop in the pilot study. Resting for a fixed time as well matters because the processor itself cools within seconds, while the heatsink and case stay warm much longer. Without this step, a run that starts on a warm laptop would look slower for reasons that have nothing to do with the strategy.

Run order. Strategies are run in rounds, and the order is reversed in alternate rounds (V0 to V3, then V3 to V0), so that slow changes during a session, such as a warming room, affect all strategies equally. Wi-Fi and cable runs are alternated within a session where possible; otherwise the session is recorded and reported with each run.

I. Correctness Checks

Speed only matters if the answers are right, so correctness is checked at four points:

1. Before any experiment, the solver is run on Pons's six test sets [5] and must return the correct result for every position.
2. The full system is tested on small boards whose results are published [6] and must reproduce them, both with and without splitting and cancelling switched on.
3. A task that is stopped or cancelled never stores a partly computed value in the transposition table, and the table is read and written safely when several threads use it at once.
4. In every measured run, the result of the start position is compared with its known value, taken from Pons's published solver [5] and confirmed by every E1 run. A run with a wrong result is discarded and reported, and its cause is fixed before testing continues.

J. Analysis

For every setting we report the mean of the three repeats together with the lowest and highest values, so that the reader can see how much the runs varied. With only three repeats, a difference smaller than this run-to-run spread is treated as unclear and is not claimed as a finding. The analysis follows the research questions:

- **RQ1 and RQ3:** solve time, speedup, combined efficiency and idle thread-time are compared step by step from V0 to V3. A timeline of each laptop's activity shows where and when time was lost, especially near the end of a run. For Laptop 1, the work done by its performance and efficient cores is compared.
- **RQ2:** each laptop's speed is plotted over time next to its clock, temperature and power. To separate the two reasons V0 can fall behind, we also work out, after the run, the best fixed split that would have been possible if every task's size had been known in advance. The gap between V0 and this best fixed split is the cost of not knowing task sizes; the rest of V0's loss against V1 to V3 is the cost of speed changing during the run.
- **RQ4:** every strategy's results on Wi-Fi are compared with those on the cable, together with the measured network delay and the share of time spent waiting for messages. Because messages are tiny and tasks are long, we expect the network to matter little at the default split depth; this expectation is tested, not assumed.
- **RQ5:** extra work, copied tasks, and lost and saved work are compared across strategies and split depths, against both the single-laptop and the unsplit references.

K. Threats to Validity

- **Two machines.** The results describe one pair of laptops; a pair with a different speed ratio may behave differently. We report each laptop's speed on its own so that readers can judge how far the findings may carry over.
- **One game and one solver.** Connect Four was chosen because its answers can be checked, but other uneven workloads may have a different spread of task sizes.
- **Operating system scheduling.** Windows decides which threads run on Laptop 1's fast and slow cores. We do not tie threads to particular cores, because we want realistic conditions, but this adds some variation between runs. The two laptops also run slightly different Windows versions, which may schedule threads differently.
- **Speed readings.** Positions searched per second also depend on the kind of positions being searched, not only on the machine. We therefore read speed changes together with the clock and power logs, never on their own.
- **Measurement load.** HWiNFO uses some processor time on both laptops, and the coordinator shares Laptop 1 with its worker. Both are present in every run, so they affect all strategies in the same way.
- **Wi-Fi changes.** A shared Wi-Fi network carries traffic from other devices and varies over time. We measure it at the start of every session and report it with the results.

- **Heat and room conditions.** The cool-down rule, the reversed run order and the recorded room temperature reduce, but cannot fully remove, the effect of heat.
- **Few repeats.** Three repeats per setting are enough to report averages and spread, but not strong statistical claims. Our conclusions stay within what the measurements show.

References

[1] T. Čížek, M. Balko, and M. Schmid, "Massively parallel proof-number search for impartial games and beyond," in *Proc. AAAI Conf. Artificial Intelligence (AAAI)*, 2026. [Online]. Available: https://arxiv.org/abs/2511.10339

[2] T.-R. Wu, H. Guei, T.-H. Wei, C.-C. Shih, J.-T. Chin, and I-C. Wu, "Game solving with online fine-tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2311.07178

[3] J. B. Fernandes, Í. A. S. de Assis, I. M. S. Martins, T. Barros, and S. Xavier-de-Souza, "Adaptive asynchronous work-stealing for distributed load-balancing in heterogeneous systems," *arXiv preprint* arXiv:2401.04494, 2024. [Online]. Available: https://arxiv.org/abs/2401.04494

[4] D. Beserra, E. D. Moreno, and J. Araujo, "Evaluating OpenMP scheduling and calibration strategies on asymmetric multicore processors: A Monte Carlo case study," in *Advanced Information Networking and Applications (AINA 2026)*, Lecture Notes on Data Engineering and Communications Technologies, vol. 294. Cham, Switzerland: Springer, 2026, pp. 357–369, doi: 10.1007/978-3-032-23260-1_30.

[5] P. Pons, "Solving Connect 4: How to build a perfect AI," 2016. [Online]. Available: http://blog.gamesolver.org/ (source code: https://github.com/PascalPons/connect4). Accessed: Oct. 7, 2026.

[6] J. Tromp, "John's Connect Four Playground." [Online]. Available: https://tromp.github.io/c4/c4.html. Accessed: Oct. 7, 2026.